



Lab Experiments

NAME: V. Vignesh

ROLL NO: 2311CS040168

SUBJECT: Agentic AI

1. Text-to-SQL Workflow

Build an end-to-end LLM workflow with retrieval and query generation

Python code:

❖ Database.py :

```
import sqlite3

# Create/connect to database
connection = sqlite3.connect("database.db")

cursor = connection.cursor()

# Create customers table
cursor.execute("""
CREATE TABLE IF NOT EXISTS customers (
    customer_id INTEGER PRIMARY KEY,
    name TEXT,
    city TEXT
)
""")

# Create orders table
cursor.execute("""
CREATE TABLE IF NOT EXISTS orders (
    order_id INTEGER PRIMARY KEY,
    customer_id INTEGER,
    order_date TEXT,
    amount REAL,
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
)
""")

# Insert customers
customers = [
    (1, "Vishnu", "Hyderabad"),
    (2, "Rahul", "Delhi"),
    (3, "Priya", "Hyderabad"),
    (4, "Arun", "Mumbai"),
    (5, "Anjali", "Bangalore")
]

cursor.executemany(
    "INSERT OR IGNORE INTO customers VALUES (?, ?, ?)",
    customers
)

# Insert orders
orders = [
    (101, 1, "2025-01-10", 5000),
    (102, 1, "2025-02-15", 3000),
    (103, 2, "2025-03-20", 7000),
    (104, 3, "2025-04-10", 4000),
    (105, 4, "2025-05-12", 9000),
    (106, 5, "2025-06-18", 6000)
]

cursor.executemany(
```

```

        "INSERT OR IGNORE INTO orders VALUES (?, ?, ?, ?)",
        orders
    )
    connection.commit()
    connection.close()

    print("Database created successfully!")

```

Note : When we run the database.py it automatically gets the database.db .

❖ Query.py :

```

import sqlite3

connection = sqlite3.connect("database.db")
cursor = connection.cursor()

query = """
SELECT c.name, SUM(o.amount) AS total_sales
FROM customers c
JOIN orders o
ON c.customer_id = o.customer_id
GROUP BY c.name
"""

cursor.execute(query)

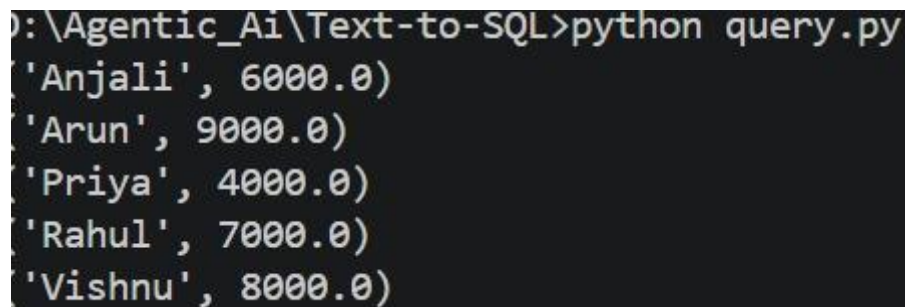
rows = cursor.fetchall()

for row in rows:
    print(row)

connection.close()

```

Output :



```

D:\Agentic_Ai\Text-to-SQL>python query.py
('Anjali', 6000.0)
('Arun', 9000.0)
('Priya', 4000.0)
('Rahul', 7000.0)
('Vishnu', 8000.0)

```

Implement indexing, retrieval, and response generation

Python code :

- ❖ Create the folder RAG – QA
- ❖ Create folder as documents in that create file as company.txt

GoComet is a logistics technology company that provides supply chain visibility solutions.

GoComet helps businesses track shipments, manage logistics operations, and improve supply chain visibility.

The company provides shipment tracking, logistics management, freight management, and analytics solutions.

Shipment tracking allows businesses to monitor the status and location of their shipments.

The platform can provide information about containers, shipments, estimated arrival times, and transportation status.

Supply chain visibility helps businesses understand where their goods are and identify potential delays.

GoComet uses technology and data to help businesses make better logistics decisions.

- ❖ pip install chromadb sentence-transformers
- ❖ Create indexing.py

```
import chromadb
from sentence_transformers import SentenceTransformer

# Load embedding model
model = SentenceTransformer("all-MiniLM-L6-v2")

# Create ChromaDB client
client = chromadb.PersistentClient(path="./chroma_db")

# Create collection
collection = client.get_or_create_collection(
    name="company_documents"
)

# Read document
with open("documents/company.txt", "r", encoding="utf-8") as file:
    text = file.read()

# Split document into chunks
chunks = [
    chunk.strip()
    for chunk in text.split("\n\n")
    if chunk.strip()
]

# Create embeddings
```

```
# Add documents to vector database
for i, chunk in enumerate(chunks):
    collection.upsert(
        ids=[f"doc_{i}"],
        documents=[chunk],
        embeddings=[embeddings[i]]
    )
print("Indexing completed!")
print("Number of chunks:", len(chunks))
```

Output :

❖ Create retrieval.py

```
question = input("Ask a question: ")
```

- ❖ Run the file.

```
D:\Agentic_AI\RAG-QA>python retrieval.py
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
Loading weights: 100%|██████████████████████████████████████████████████████████████████████████████| 103/103 [00:00<00:00, 4030.24it/s]
Ask a question: What does GoComet provide?

Retrieved information:
```

- GoComet is a logistics technology company that provides supply chain visibility solutions.
- GoComet helps businesses track shipments, manage logistics operations, and improve supply chain visibility.
- GoComet uses technology and data to help businesses make better logistics decisions.

```
response = requests.post(
    "http://localhost:11434/api/generate",
    json={
        "model": "llama3.2",
        "prompt": prompt,
        "stream": False
    }
)
```

```
result = response.json()

return result["response"]
```

- ```
from retrieval import retrieve
from generate import generate_answer

question = input("Ask a question: ")

Retrieve relevant documents
documents = retrieve(question)

Combine retrieved documents
context = "\n\n".join(documents)

Generate answer
answer = generate_answer(
 question,
 context
)

print("\nAnswer:")
print(answer)
```

```
D:\Agentic_AI\RAG-QA>python main.py
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
Loading weights: 100%|██| 103/103 [00:00<00:00, 6563.86it/s]
Ask a question: What is shipment tracking?

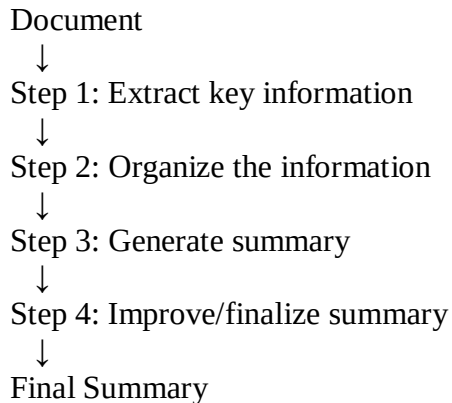
Answer:
Based on the provided context, I would say that shipment tracking refers to the process of monitoring the status and location of a business's shipments.
```

- ```
graph TD;
    Question --> Embedding;
    Embedding --> ChromaDB;
    ChromaDB --> Retrieve[Retrieve relevant chunks];
    Retrieve --> Context;
    Context --> Ollama;
    Ollama --> Answer;
```

3. Prompt Chaining for Summarization

Experiment with multi-step prompt pipelines

❖ Prompt Chaining :



❖ Create the folder as Prompt-Chaining

❖ Create the file as document.txt

Artificial intelligence is transforming many industries. Companies use AI to automate repetitive tasks, analyze large amounts of data, improve customer service, and support decision making.

Generative AI has become particularly important because it can create text, images, code, and other forms of content. Large language models can understand natural language and generate human-like responses.

However, organizations also face challenges when adopting AI. These include data privacy, security, model reliability, high computational costs, and the need for skilled employees.

Successful AI adoption requires organizations to combine technology with appropriate governance, security practices, human oversight, and continuous evaluation.

❖ Install - pip install requests

❖ Create main.py

```
import requests

MODEL = "llama3.2"

def ask_llm(prompt):
    response = requests.post(
        "http://localhost:11434/api/generate",
        json={
            "model": MODEL,
            "prompt": prompt,
            "stream": False
        }
    )

    response.raise_for_status()
```



```

    return response.json()["response"]

# -----
# Read document
# -----

with open("document.txt", "r", encoding="utf-8") as file:
    document = file.read()

# -----
# STEP 1: Extract key information
# -----

prompt1 = f"""
Read the following document.

Extract the most important facts, ideas, problems,
and conclusions.

Do not write a summary yet.

Document:
{document}
"""

key_information = ask_llm(prompt1)
print("\n==== STEP 1: KEY INFORMATION =====")
print(key_information)

# -----
# STEP 2: Organize information
# -----

prompt2 = f"""
Organize the following information into clear categories.
Use these categories:

1. Main Topic
2. Benefits
3. Challenges
4. Important Concepts
5. Conclusion
Information:
{key_information}
"""

organized_information = ask_llm(prompt2)

print("\n==== STEP 2: ORGANIZED INFORMATION =====")
print(organized_information)

# -----
# STEP 3: Generate summary
# -----

prompt3 = f"""
Write a concise summary using the information below.

Requirements:
- 100 to 150 words

```

- Include the main topic
- Include important benefits
- Include important challenges
- Include the conclusion
- Do not add information that is not provided

Information:

```
{organized_information}
```

```
"""
```

```
summary = ask_llm(prompt3)
print("\n===== STEP 3: SUMMARY =====")
print(summary)
```

```
# -----
```

```
# STEP 4: Improve summary
```

```
# -----
```

```
prompt4 = f"""
```

```
Improve the following summary.
```

```
Requirements:
```

- Keep the original meaning
- Remove unnecessary repetition
- Make it clear and easy to understand
- Keep it between 100 and 150 words
- Do not introduce new facts

```
Summary:
```

```
{summary}
```

```
"""
```

```
final_summary = ask_llm(prompt4)
```

```
print("\n===== STEP 4: FINAL SUMMARY =====")
```

```
print(final_summary)
```

Output :

```
D:\Agentic_AI\Prompt-Chaining>python main.py

===== STEP 1: KEY INFORMATION =====
Here are the most important facts, ideas, problems, and conclusions from the document:

**Important Facts:**

* Artificial intelligence (AI) is transforming many industries.
* Generative AI can create text, images, code, and other forms of content.
* Large language models can understand natural language and generate human-like responses.

**Ideas:**

* Organizations use AI to automate tasks, analyze large amounts of data, improve customer service, and support decision making.
* The importance of generative AI in creating various types of content.

**Problems:**

* Data privacy
* Security
* Model reliability
* High computational costs
* Need for skilled employees

**Conclusions:**

* Successful AI adoption requires a combination of technology, governance, security practices, human oversight, and continuous evaluation.

===== STEP 2: ORGANIZED INFORMATION =====
Here is the organized information with the specified categories:
```

4. SQL Agent with Tool Use

Develop a ReAct-based agent using database tools

❖ Objective

Build a ReAct-based SQL Agent that can:

1. Understand a user's question
2. Decide which database tool to use
3. Inspect the database
4. Generate SQL
5. Execute SQL
6. Observe the result
7. Give the final answer

❖ Reuse your SQLite database

- We already have database.db

❖ Create tools.py

```
import sqlite3

DATABASE = "database.db"

def list_tables():
    """Return all tables in the database."""

    connection = sqlite3.connect(DATABASE)
    cursor = connection.cursor()

    cursor.execute("""
        SELECT name
        FROM sqlite_master
        WHERE type='table'
        AND name NOT LIKE 'sqlite_%'
    """)

    tables = [row[0] for row in cursor.fetchall()]

    connection.close()

    return tables

def get_schema(table_name):
    """Return the schema of a table."""

    connection = sqlite3.connect(DATABASE)
    cursor = connection.cursor()

    cursor.execute(f"PRAGMA table_info({table_name})")

    columns = cursor.fetchall()
    connection.close()
```

```

if not columns:
    return f"Table '{table_name}' does not exist."

schema = []

for column in columns:
    schema.append({
        "name": column[1],
        "type": column[2]
    })
return schema

def execute_sql(query):
    """Execute a read-only SQL query."""

    query = query.strip()

    # Security: allow only SELECT
    if not query.upper().startswith("SELECT"):
        return "ERROR: Only SELECT queries are allowed."

    try:

        connection = sqlite3.connect(DATABASE)
        cursor = connection.cursor()

        cursor.execute(query)

        rows = cursor.fetchall()
        columns = [
            description[0]
            for description in cursor.description
        ]
        connection.close()
        return {
            "columns": columns,
            "rows": rows
        }
    except Exception as e:
        return f"SQL ERROR: {str(e)}"

```

❖ Test the tools first

- Create test_tools.py

```

from tools import list_tables, get_schema, execute_sql
print("TABLES:")
print(list_tables())

print("\nCUSTOMERS SCHEMA:")
print(get_schema("customers"))

print("\nSQL RESULT:")
print(
    execute_sql(
        "SELECT * FROM customers"
    )
)

```

Run

```
D:\Agentic_Ai\SQL-Agent>python test_tools.py
TABLES:
['customers', 'orders']

CUSTOMERS SCHEMA:
[{'name': 'customer_id', 'type': 'INTEGER'}, {'name': 'name', 'type': 'TEXT'}, {'name': 'city', 'type': 'TEXT'}]

SQL RESULT:
{'columns': ['customer_id', 'name', 'city'], 'rows': [(1, 'Vishnu', 'Hyderabad'), (2, 'Rahul', 'Delhi'), (3, 'Priya', 'Hyderabad'), (4, 'Arun', 'Mumbai'), (5, 'Anjali', 'Bangalore')]}
```

❖ Create the ReAct agent

- Now create agent.py

```
import requests

from tools import (
    list_tables,
    get_schema,
    execute_sql
)

MODEL = "llama3.2"

def ask_llm(prompt):

    response = requests.post(
        "http://localhost:11434/api/generate",
        json={
            "model": MODEL,
            "prompt": prompt,
            "stream": False
        }
    )
    response.raise_for_status()

    return response.json()["response"]

def agent(question):

    print("\nUSER:")
    print(question)

    # -----
    # STEP 1: Get available tables
    # -----

    tables = list_tables()

    print("\nTOOL: list_tables()")
    print("RESULT:", tables)
    # -----
    # STEP 2: Get schemas
    # -----

    schemas = {}
```

```

for table in tables:

    schemas[table] = get_schema(table)

    print(f"\nTOOL: get_schema({table})")
    print("RESULT:", schemas[table])

# -----
# STEP 3: Ask LLM to generate SQL
# -----

prompt = f"""
You are a ReAct SQL agent.

You have access to a SQLite database.

Tables:
{tables}

Schemas:
{schemas}

User question:
{question}

Think about which tables and columns are needed.

Then generate ONE SQLite SELECT query.

Rules:
- Only generate SELECT queries.
- Do not INSERT.
- Do not UPDATE.
- Do not DELETE.
- Do not DROP.
- Use only the tables and columns provided.

Return ONLY the SQL query.
"""

sql = ask_llm(prompt).strip()

# Remove markdown code fences if the LLM adds them
sql = sql.replace("```sql", "")
sql = sql.replace("```", "")
sql = sql.strip()

print("\nAGENT ACTION:")
print(sql)

# -----
# STEP 4: Execute SQL
# -----
result = execute_sql(sql)
print("\nTOOL: execute_sql()")

print("RESULT:", result)

# -----
# STEP 5: Generate final answer

```

```

# -----

final_prompt = f"""
You are a helpful database assistant.

User question:
{question}

SQL query:
{sql}

Database result:
{result}

Answer the user's question using the database result.

Do not mention internal tools.
Give a short and clear answer.
"""

answer = ask_llm(final_prompt)
return answer

```

❖ Create main.py

```

from agent import agent

question = input("Ask a database question: ")

answer = agent(question)

print("\nFINAL ANSWER:")
print(answer)

```

❖ Run the agent

Output :

```

D:\Agentic_Ai\SQL-Agent>python main.py
Ask a database question: What is the total sales of Vishnu?

USER:
What is the total sales of Vishnu?

TOOL: list_tables()
RESULT: ['customers', 'orders']

TOOL: get_schema(customers)
RESULT: [{'name': 'customer_id', 'type': 'INTEGER'}, {'name': 'name', 'type': 'TEXT'}, {'name': 'city', 'type': 'TEXT'}]

TOOL: get_schema/orders)
RESULT: [{'name': 'order_id', 'type': 'INTEGER'}, {'name': 'customer_id', 'type': 'INTEGER'}, {'name': 'order_date', 'type': 'TEXT'}, {'name': 'amount', 'type': 'REAL'}]

AGENT ACTION:
SELECT SUM(o.amount) FROM orders o JOIN customers c ON o.customer_id = c.customer_id WHERE c.name = 'Vishnu'

TOOL: execute_sql()
RESULT: {'columns': ['SUM(o.amount)'], 'rows': [(8000.0,)]}

FINAL ANSWER:
The total sales of Vishnu is $8,000.

```